Class Imbalance and RBF-Networks

Contents

7.1 Dealing with imbalanced data **1**
7.1.1 Motivation . . . 1
7.1.2 Confusion matrix . . . 2
7.1.3 Performance metrics . . . 2

7.2 Fairness in MI **3**
7.2.1 Aspects . . . 4
7.2.2 Include aspects of fairness . . . 4
7.2.3 Fairness via Model Construction . . . 4
7.2.4 Further considerations . . . 5
7.2.4.1 Calibration . . . 5

7.3 Non-parametric classification **7**
7.3.1 The setting . . . 7
7.3.2 k nearest neighbor classifier . . . 7
7.3.2.1 For multi-class classification . . . 8
7.3.2.2 the binary classification case . . . 8
7.3.3 Parzen window classification . . . 10

7.4 RBF-Networks **12**
7.4.1 Recap: Regression on transformed data using nonlinear basis functions . . . 12
7.4.2 Nonlinear basis function . . . 12
7.4.3 Setting for binary classification/ scalar regression . . . 12
7.4.4 Multi-dimensional RBF-Networks . . . 13
7.4.5 RBF-Network vs. Parzen window classification . . . 14
7.4.6 MLPs vs. RBF-Networks . . . 16

7.1 Dealing with imbalanced data

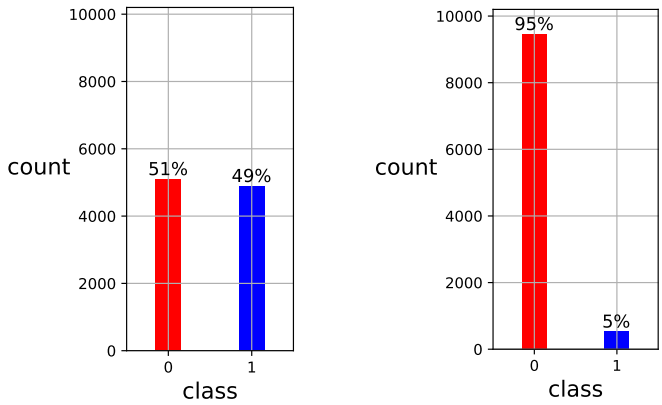
7.1.1 Motivation

Proportion of class labels in the dataset is not uniform. Not the same as "easy vs. difficult" classes. This is only about class frequency.

A strong imbalance leads to the classifier learning a trivial solution that indeed minimizes the average cost over the training samples.

$$E^T = \frac{1}{p}\sum_{\alpha=1}^{p} e^{(\alpha)} \tag{7.1.1}$$

$$E^T = \frac{1}{p}\Big[\sum_{\beta\in D_+}^{|D_+|} e^{(\beta)} + \sum_{\beta\in D_-}^{|D_-|} e^{(\beta)}\Big] \tag{7.1.2}$$



(a) balanced $\approx 1:1$ (b) highly unbalanced

where set D_+ and D_- are the subsets of data of only positive and negative samples, respectively:

$$D_+ := \left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{y}}_T^{(\alpha)} \right) \middle| \, y_T^{(\alpha)} > 0 \right\} \quad \text{(subset of positive samples)} \tag{7.1.3}$$

and

$$D_- := \left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{y}}_T^{(\alpha)} \right) \middle| \, y_T^{(\alpha)} \leq 0 \right\} \quad \text{(subset of positive samples)} \tag{7.1.4}$$

By always predicting "0" it will be correct 95% of the time.

7.1.2 Confusion matrix

The confusion matrix differentiates between the types of mistakes a classifier makes.

		Ground truth label	
		Positive	Negative
		"Cat"	"not Cat"
Prediction	Positive	**T**rue **P**ositives	**F**alse **P**ositives
	Negative	**F**alse **N**egatives	**T**rue **N**egatives
		P	N

7.1.3 Performance metrics

$$\text{fraction of correct classifications} \mathrel{\widehat{=}} \text{Accuracy} = \frac{TP + TN}{P + N} \tag{7.1.5}$$

In our imbalanced example:

$$\frac{TP + TN}{P + N} = \frac{0 + 9500}{500 + 9500} = 0.95 \tag{7.1.6}$$

Accuracy is a misleading metric when the classes are imbalanced.

Other metrics:

How many of the positive labels did we get:

$$\text{sensitivity (recall)} = \frac{TP}{P} \tag{7.1.7}$$

and how often were we correct, whenever we predicted positive?

$$\text{precision} = \frac{TP}{TP + FP} \tag{7.1.8}$$

Recall and precision can be combined into:

$$\text{F1 score} = 2 \cdot \frac{\text{recall} \times \text{precision}}{\text{recall} + \text{precision}} \tag{7.1.9}$$

Similarly, for the negative class. How many of the negative class did we get:

$$\text{specificity} = \frac{TN}{N} \tag{7.1.10}$$

Recall and specificity can be combined into:

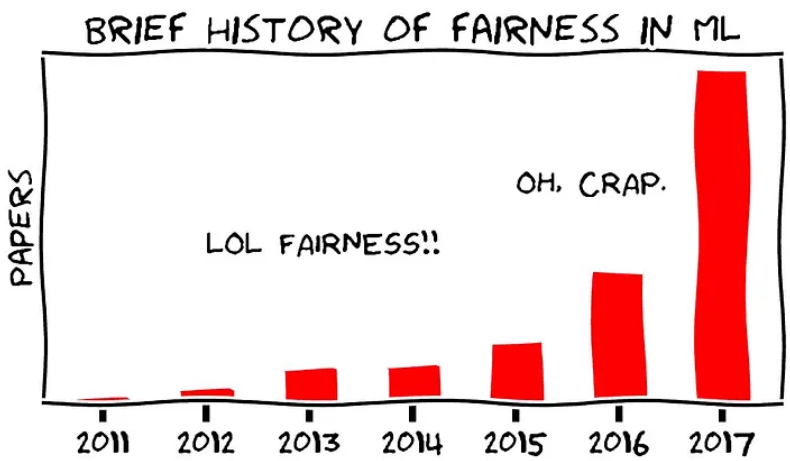
$$\text{Balanced Accuracy} = \frac{1}{2}(\text{recall} + \text{specificity}) \tag{7.1.11}$$

"F1 score" and "Balanced Accuracy" are common choices for assessing performance of a classifier evaluated on imbalanced data with different implications with regards to how "consistent" this imbalance is.

7.2 Fairness in MI

Short Excursion

Aspects of Fairness in MI



[1]

[1] image taken from towardsdatascience.com

7.2.1 Aspects

1. legal aspects (e.g. to help identify discriminating features)
2. data representational of truth
3. ensure, data samples aren't
   a) *skewed*, i.e. future observations confirm prediction ("confirmation bias") *e.g.* less crime reported → less police → less crime detected → less crime reported
   b) *tainted*, i.e. bias existing in old data is kept *e.g.* less female/non-binary professors in the past → MI decides for new position in favour of male candidates
   c) *limited*, i.e. less reliable or fewer data samples of minority groups *e.g.* carcinoma detection on skin → less accuracy in minority group
   d) *proxies* i.e. delete sensible features leaves "proxies"/"hidden features" that MI learns *e.g.* exclude gender in CVs, parental leave duration remains

7.2.2 Include aspects of fairness

(a) Model construction[2]

(b) Preprocessing[3]

(c) Optimization[4]

7.2.3 Fairness via Model Construction

Setting: Binary classification - hiring for a job, one single sensitive attribute

- $\{\underline{\mathbf{x}}^{(\alpha)} \in \mathbb{R}^N\}_{\alpha=1}^p$; e.g. entries for education, experience, grades, ...
- $y_T^{(\alpha)} \in 0, 1$; e.g. hire/no-hire
- $\{a^{(\alpha)} \in \{0, 1\}\}_{\alpha=1}^p$; sensitive attribute per applicant
- $C(\underline{\mathbf{x}}, a)$ as binary predictor.

a. **Unawareness**: exclude sensitive attribute in training data

b. **Demographic parity**: ensure that probabilities of (non-)hiring are the same for both groups

$$P_{a=0}(C(\underline{\mathbf{x}}, a) = y) = P_{a=1}(C(\underline{\mathbf{x}}, a) = y),\ y \in \{0, 1\}$$

c. **Equalized odds**: ensure same conditional probabilities

$$P_{a=0}(C = y | y_T = \hat{y}) = P_{a=1}(C = y | y_T = \hat{y}),\ \forall y, \hat{y} \in \{0, 1\}$$

[2] e.g. see Learning Fair Representations by Zemel et al.

[3] e.g. see iFair: Learning Individually Fair Data Representations for Algorithmic Decision Making, by Lahoti et al.

[4] e.g. see Fairness Beyond Disparate Treatment & Disparate Impact: Learning Classification without Disparate Mistreatment, by Zafar et al.

7.2.4 Further considerations

1. Is there only one recall value for my classifier that predicts with $y(\underline{\mathbf{x}}) \in [0, 1]$ (or $[-1, 1]$, $(0, 1)$)?
2. What if the proportions in my training set is different from that of the validation/test set?

7.2.4.1 Calibration

Calibrating the classifier:

Finding an operating point by adjusting the threshold that converts the prediction of classifier into a hard decision.

General procedure:

1. train binary classifier (assuming $y(\underline{\mathbf{x}}) \in [0, 1]$)
2. make predictions on a *hold-out* set (selecting an operating point counts as a hyper-parameter selection)
3. Save "probabilistic" output, no thresholding.
4. for threshold $\mathrm{thr} \in [0, 1]$ do:
   a) assign predictions to classes using current threshold thr
   b) compute confusion matrix (e.g. TP and FP)

   repeat
5. Compute metrics as a function of thr_i

Q 7.2.1: *What does thr effectively represent?* - Effectively the bias of the output neuron. Changing thr corresponds to shifting/translating the hyperplane that divides the two classes. The orientation of the hyperplane remains the same.

Example for the calibration procedure: *Receiver operating characteristics (ROC)*. Keep track of TP and FP as a function of different threshold values.

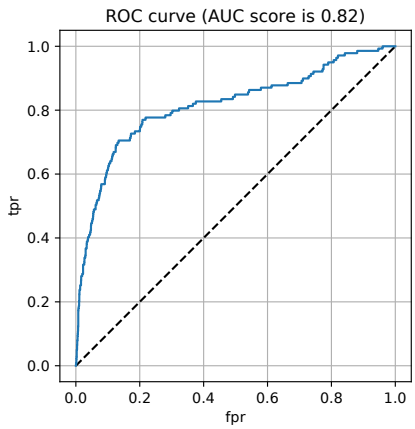


Figure 7.2.1: ROC curve. $TP/P \mathrel{\widehat{=}}$ true positive rate (TPR) and $FP/N \mathrel{\widehat{=}}$ false positive rate (FPR)

The higher the area under the curve (ROC AUC) the better the classifier.

Q 7.2.2: *What does the dashed diagonal line represent?*

But at the end of the day I need to pick an operating point (i.e. one threshold value).

Q 7.2.3: *What criterion could I use to favor one threshold over another?*

- Assign a "decision cost $\$_{ij}$" for different classes where $\$_{ij}$ denotes the cost of predicting class C_i when the true label is C_j (cf. lecture slides 1.4)

What if the proportions in my training set are different from those of the validation/test set? Which metrics are robust to changes in proportions between different splits of the data.[5]

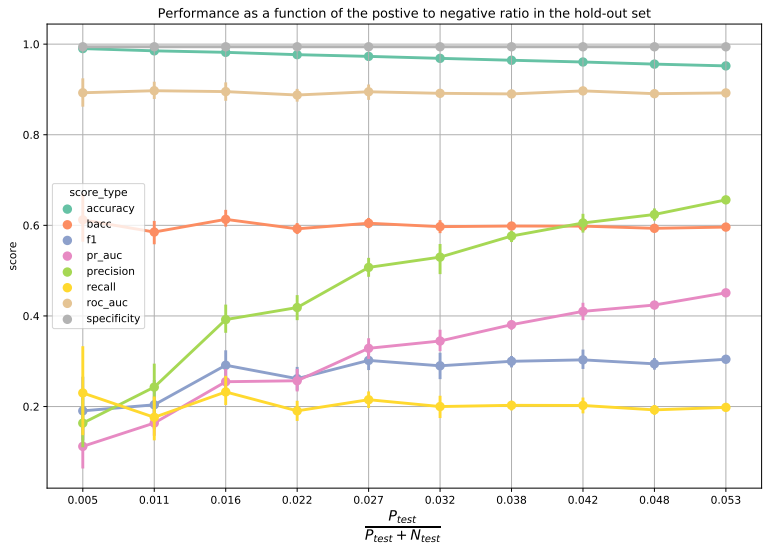


Figure 7.2.2: Comparing metrics under different conditions

All we did so far is measure generalization performance and tune our operating point.

Q 7.2.4: *Is there anything else we can do about imbalanced data?*

1. Train the classifier on a synthetically balanced dataset by
   - sub-sampling from the majority class (no one likes to throw away data)
   - oversampling the minority class

   Q 7.2.5: *Any downsides to oversampling?*

   - the resulting dataset could be too large.
2. class weighted loss function: e.g. weighted cross entropy:

$$e^{(\alpha)} = -\gamma\, y_T^{(\alpha)} \cdot \ln[y^{(\alpha)}] - \beta\,(1 - y_T^{(\alpha)}) \cdot \ln[1 - y^{(\alpha)}] \tag{7.2.1}$$

   where β and γ represent "class weights"

[5] cf. jupyter notebook on ISIS for a comparison of different metrics under different conditions.

7.3 Non-parametric classification

7.3.1 The setting

Specifying the data and model for a multi-class classification with M classes (i.e. M-way classification)

Data:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{y}}_T^{(\alpha)} \right) \right\}_{\alpha=1}^{p}$$

where $\underline{\mathbf{y}}_T^{(\alpha)} \in \{0,1\}^M$ with $\sum_{c=1}^{M} (y_T)_c = 1$ (one-hot encoding of class labels)

Model:

$$\underline{\mathbf{y}}(\underline{\mathbf{x}}) \in \mathbb{R}^M$$

with $\sum_{c=1}^{M} y_c(\underline{\mathbf{x}}) = 1$ and $y_c(\underline{\mathbf{x}}) \geq 0 \; \forall \, c$[2mm]

Predictions $\underline{\mathbf{y}}(\underline{\mathbf{x}})$ can be used to give

- probabilities of the predicted class (e.g. $y_5(\underline{\mathbf{x}}) = 0.75 \quad \rightsquigarrow$ class "5" with 75% probability.
- hard decisions:

$$\underset{c=1,\ldots,M}{\operatorname{argmax}} \; y_c(\underline{\mathbf{x}}) \tag{7.3.1}$$

7.3.2 k nearest neighbor classifier

Prediction follows an *electoral committee*; the majority vote of the k nearest neighbors around a query point.

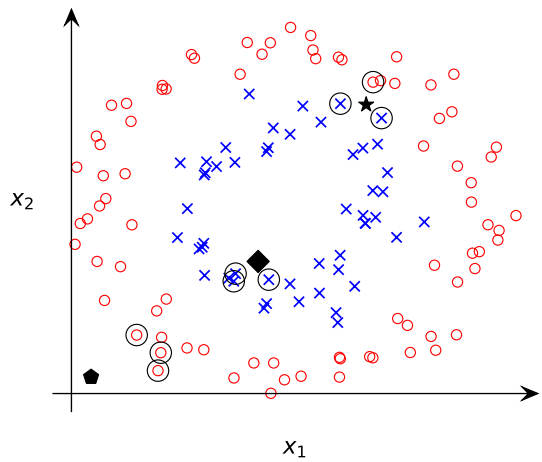


Figure 7.3.1: kNN with $k = 3$

Let $k\mathrm{NN}(\underline{\mathbf{x}})$ be the indices $\{\beta_1, \beta_2, \ldots, \beta_k\}$ of the k data points closest to $\underline{\mathbf{x}}$ w.r.t. Euclidean norm:

$$\beta_j = \underset{\substack{\alpha \in \{1,\ldots,p\} \setminus \\ \{\beta_1,\ldots,\beta_{j-1}\}}}{\operatorname{argmin}} \; \|\underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{x}}\|_2 \tag{7.3.2}$$

7.3.2.1 For multi-class classification

For multi-class classification the kNN classifier (for fixed $k \mathrel{\widehat{=}}$ hyper-parameter) is defined by:

$$\underline{\mathbf{y}}(\underline{\mathbf{x}}) = \frac{1}{k} \sum_{\beta \in k\mathrm{NN}(\underline{\mathbf{x}})} \underline{\mathbf{y}}_T^{(\beta)} \tag{7.3.3}$$

$\underline{\mathbf{y}}(\underline{\mathbf{x}})$ in Eq. (7.3.3) is effectively an arithmetic average of the labels in the neighborhood.

Example $k = 3$, $M = 4$:

$$\underline{\mathbf{y}}(\underline{\mathbf{x}}) = \frac{1}{3}\left[\begin{pmatrix}1\\0\\0\\0\end{pmatrix} + \begin{pmatrix}1\\0\\0\\0\end{pmatrix} + \begin{pmatrix}0\\1\\0\\0\end{pmatrix}\right] = \frac{1}{3}\begin{pmatrix}2\\1\\0\\0\end{pmatrix} = \begin{pmatrix}2/3\\1/3\\0\\0\end{pmatrix} \tag{7.3.4}$$

applying a hard decision would yield $c = 1$.

7.3.2.2 the binary classification case

For binary classification, we can set $M = 2$ for a multi-class kNN classifier.

Alternatively, we can construct the kNN classifier for binary classification using *scalar* labels:

$$y_T \in \{0, 1\} \quad \text{and} \quad y(\underline{\mathbf{x}}) \in \mathbb{R} \tag{7.3.5}$$

with

$$y(\underline{\mathbf{x}}) \geq 0 : \begin{cases} y(\underline{\mathbf{x}}) & \text{probability of class } 1 \\ 1 - y(\underline{\mathbf{x}}) & \text{probability of class } 0 \end{cases} \tag{7.3.6}$$

Q 7.3.1: *What is the role of k in terms of model complexity? $k = 1$ vs. $k = p$*
Hint: Jitter the data (or the query points, but not both) and see which choice

of k leads to different (more noisy) predictions.

Q 7.3.2: *How would you programatically find the k nearest neighbors for some query point $\underline{\mathbf{x}}$?*

- The k nearest neighboars can be found by

1. computing the Euclidean distance between the query point and all training points,
2. sorting the distances in ascending order
3. selecting the first k points

Remarks:

1. The value of k controls the model complexity ($k = 1 \rightsquigarrow$ over-fitting, larger $k \rightsquigarrow$ under-fitting)

2. Selecting the k nearest neighbors of some point $\underline{\mathbf{x}}$ can be costly but can be made more efficient using data structures such as *Kdtrees*)

7.3.3 Parzen window classification

Prediction is based on

- <u>all</u> data points (no longer restricted to the k nearest neighbors)
- and the contribution of each point is weighted according to some window/"kernel" function $\kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}')$ evaluated for a pair of points $\underline{\mathbf{x}}$ and $\underline{\mathbf{x}}'$.

$$\underline{\mathbf{y}}(\underline{\mathbf{x}}) := \frac{1}{Z} \sum_{\alpha=1}^{p} \underline{\mathbf{y}}_T^{(\alpha)} \kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)}) \tag{7.3.7}$$

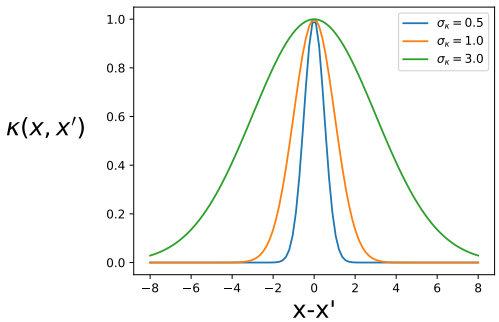
with

$$Z := \sum_{\alpha=1}^{p} \kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}^{(\alpha)}) \tag{7.3.8}$$

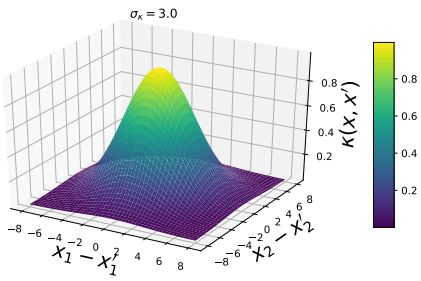
An example for the window/"kernel" function $\kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}')$ would be the Gaussian function:

$$\kappa(\underline{\mathbf{x}}, \underline{\mathbf{x}}') := \exp\left(-\frac{\|\underline{\mathbf{x}} - \underline{\mathbf{x}}'\|_2^2}{2\,\sigma_\kappa^2}\right) \tag{7.3.9}$$

where σ_κ is referred to as the *width* of the kernel.



(a) For data in 1D

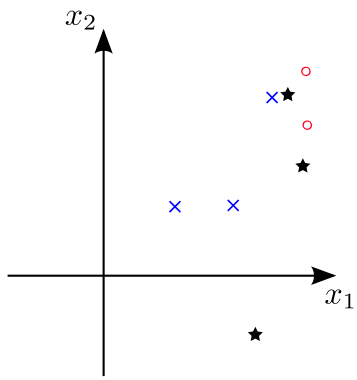


(b) For data in 2D

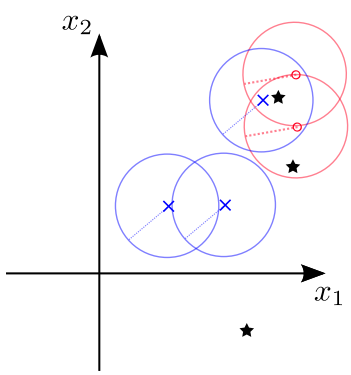
Figure 7.3.2: The Gaussian kernel function

Binary classification example with 2D data:

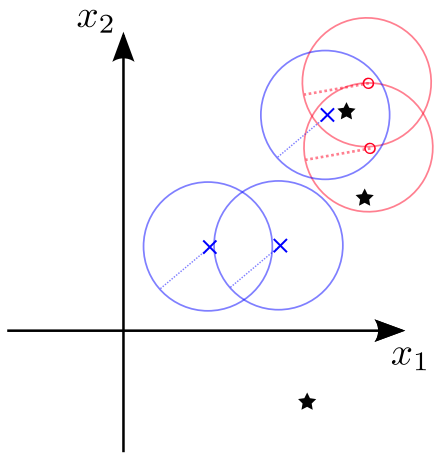
Q 7.3.3: *What is the role of the kernel width σ_κ in terms of model complexity?*



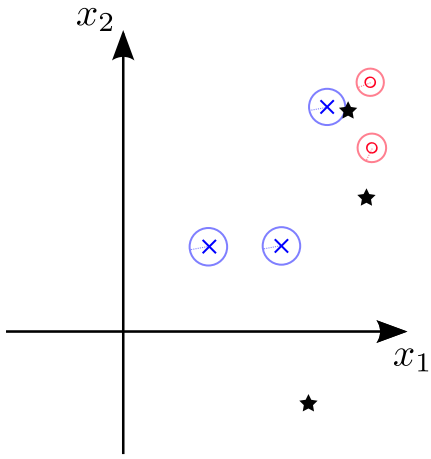
(a) Binary classification data in 2D



(b) A Gaussian window around each training point



(a) wide kernel width



(b) narrow kernel width

Hint: Jitter the query points (or the data) and see which choice of σ_κ will lead to different (more noisy) predictions.

Remarks:

1. The width of the kernel κ controls the model complexity (narrow width $\rightsquigarrow$ over-fitting (high variance), wide $\rightsquigarrow$ under-fitting)

   One way to go about this is to apply a small noise to the query points (i.e. jitter) and see how the predictions will vary with different σ_κ. If this small noise in the position of the points leads to different predictions we are basically looking at high variance.

2. Using all data points. Potentially costly with large p

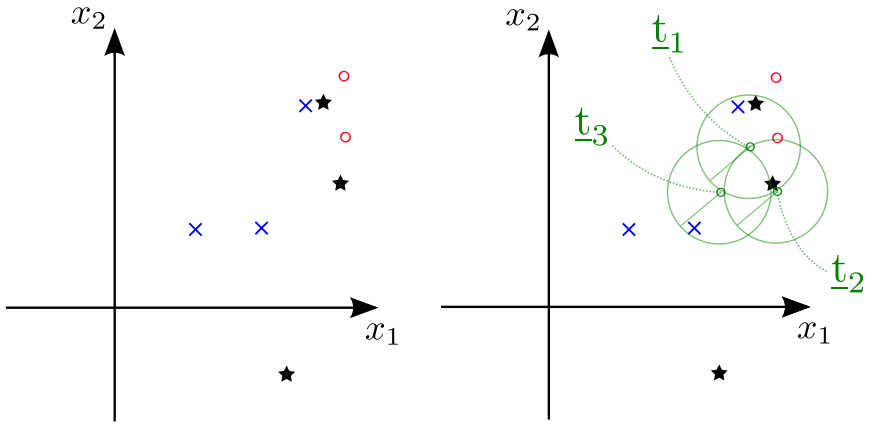
7.4 RBF-Networks

A trade-off between kNN classification and Parzen window classification.

Instead of taking all p points to predict the class probabilities (Parzen), limit the prediction to $k \ll p$ ~~points~~ "representative" functions.

Represent all points $\underline{\mathbf{x}}$ in terms of basis functions $\phi_i(\underline{\mathbf{x}}), i = 1 \ldots, k$

Let k be the number of basis functions.



(a) Two-class data in 2D (b) $k = 3$ "representatives"

7.4.1 Recap: Regression on transformed data using nonlinear basis functions

7.4.2 Nonlinear basis function

Feature transformation ($\underline{\mathbf{x}} \in \mathbb{R}^N$):

$$\underline{\phi} : \underline{\mathbf{x}} \mapsto \underline{\phi}(\underline{\mathbf{x}}) \tag{7.4.1}$$

$\phi(\underline{\mathbf{x}})$ can transform our N-dim input $\underline{\mathbf{x}}$ into the feature space spanned by the k basis functions positioned at $\underline{\mathbf{t}}_1, \ldots, \underline{\mathbf{t}}_k$.

Assume $\{\underline{\mathbf{t}}_i\}$ and corresponding kernel width $\{\sigma_i\}$ have already been determined.

Transform the feature space using *radial basis functions* (Gaussian kernel functions):

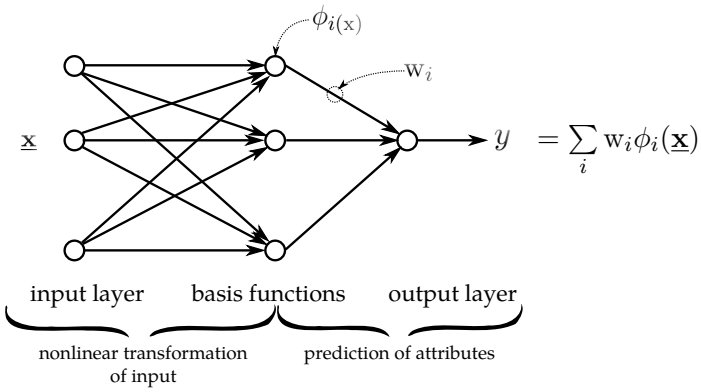
$$\phi_i(\underline{\mathbf{x}}) \in \exp\left(-\frac{\|\underline{\mathbf{x}} - \underline{\mathbf{t}}_i\|_2^2}{2\,\sigma_i^2}\right), \quad i = 1, \ldots, k \tag{7.4.2}$$

7.4.3 Setting for binary classification/ scalar regression

<u>Data</u>:

<u>Model</u>:

$$\begin{aligned}
\text{transformed input} \quad & \underline{\phi}^{(\alpha)} := \underline{\phi}(\underline{\mathbf{x}}^{(\alpha)}) = \big(\underbrace{1}_{\phi_0}, \phi_1(\underline{\mathbf{x}}^{(\alpha)}), \ldots, \phi_k(\underline{\mathbf{x}}^{(\alpha)})\big)^\top \\
& \Rightarrow \underline{\phi}^{(\alpha)} \in \mathbb{R}^{k+1} \\
\text{label} \quad & y_T^{(\alpha)} \in \mathbb{R} \\
\text{with} \quad & \alpha = 1, \ldots, p
\end{aligned}$$



The output is a linear neuron on the *the transformed* input:

$$y(\underline{\phi}; \underline{\mathbf{w}}) = \sum_{i=0}^{k} w_i \phi_i(\underline{\mathbf{x}}) = \underline{\mathbf{w}}^\top \underline{\phi} \tag{7.4.3}$$

Start from $i = 0$ to include bias node (unlike the case with monomials where the bias was already present at $i = 1$). Set $\underline{\mathbf{\Phi}} = \left(\underline{\phi}^{(1)}, \ldots, \underline{\phi}^{(\alpha)}, \ldots, \underline{\phi}^{(p)}\right) \in \mathbb{R}^{k+1,p}$

quadratic cost function. Reuse solution for linear regression:

$$\underline{\mathbf{w}}^* = \left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top\right)^{-1} \underline{\mathbf{\Phi}}\,\underline{\mathbf{y}}_{\text{True}}^\top \tag{7.4.4}$$

Q 7.4.1: *What about using RBF-Networks for classification?*

- The quadratic cost function can still be applied to learning the parameters in an RBF-Network for solving classification problems as well but the performance is often less than when using cross-entropy loss.

7.4.4 Multi-dimensional RBF-Networks

Generalizing nonlinear basis functions in linear models:

$$\underline{\mathbf{y}}(\underline{\phi}(\underline{\mathbf{x}})) = \begin{pmatrix} y_1(\underline{\phi}(\underline{\mathbf{x}})) \\ y_2(\underline{\phi}(\underline{\mathbf{x}})) \\ \vdots \\ y_M(\underline{\phi}(\underline{\mathbf{x}})) \end{pmatrix} = \underline{\mathbf{W}}^\top \underline{\phi}(\underline{\mathbf{x}}) \tag{7.4.5}$$

with $\underline{\mathbf{W}} \in \mathbb{R}^{k+1,M}$

closed form solution for quadratic cost:

$$\underline{\mathbf{W}}^* = \underbrace{\left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top\right)^{-1}}_{\substack{\text{assume}\\ \text{invertibility}}} \underline{\mathbf{\Phi}}\,\underline{\mathbf{Y}}_{\text{True}}^\top \tag{7.4.6}$$

Q 7.4.2: *What is $\underline{\mathbf{Y}}_{True}$? What is the shape of this matrix?*

where $\underline{\mathbf{Y}}_{\text{True}} := \left(\underline{\mathbf{y}}_{\text{True}}^{(1)}, \ldots, \underline{\mathbf{y}}_{\text{True}}^{(p)}\right) \in \mathbb{R}^{M,p}$

Remarks:

- RBF-Networks are universal function approximators like MLPs.
- RBF-Networks suffer from the "curse of dimensionality"
- Regularization is needed:

  For weight decay:

$$\underline{\mathbf{W}}^* = \left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top + \lambda\,\underline{\mathbf{I}}_{k+1}\right)^{-1} \underline{\mathbf{\Phi}}\,\underline{\mathbf{Y}}_{\text{True}}^\top \tag{7.4.7}$$

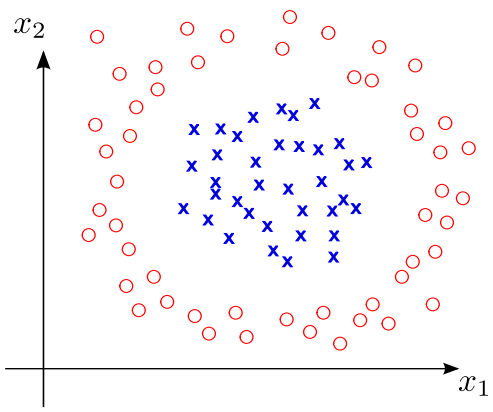
7.4.5 RBF-Network vs. Parzen window classification

Q 7.4.3: *Which model class is a generalization of the other?*

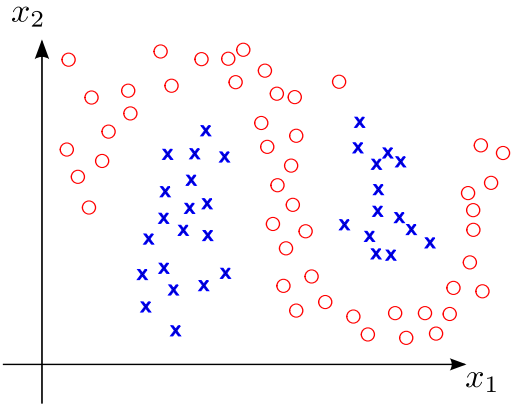
Q 7.4.4: *How would you construct an RBF-Network to mimic a Parzen window classifier?*

- An RBF-Network can be constructed and parameterized to act as a Parzen window classifier by placing a "representative'/centroid over every point $^{(\alpha)}$.

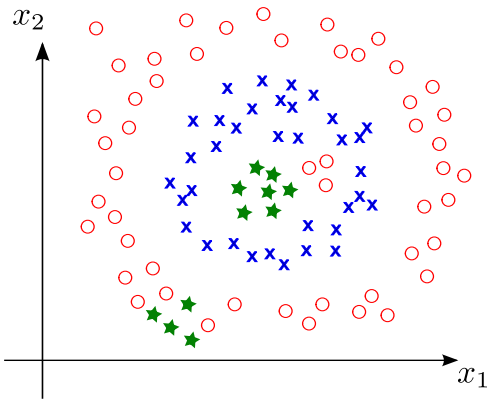
Q 7.4.5: *Can an RBF-Network solve this problem? How?*



Q 7.4.6: *Can an RBF-Network solve this problem? How many basis functions does it need?*



Q 7.4.7: *Can an RBF-Network solve this problem using only 2 centroids?*



7.4.6 MLPs vs. RBF-Networks

cf. Haykin 5.11

MLPs	RBF-Networks
global approximation	local approximation due to exponentially decaying non-linearity
+ better performance than RBF with same # of parameters	- less performance than MLP with same # of parameters
+ usually better performance with larger datasets	+ suitable for small datasets
- data hungry	+ faster training with small # of basis functions
+ adapts functions to data and labels	can benefit from unlabeled data (semi-supervised learning)
common neuronal mode between hidden and output neurons	The hidden layer serves a very different purpose than the output layer
total input of a hidden neuron determined via inner product $\underline{\mathbf{w}}_i^\top \underline{\mathbf{x}}$	total input is based on a Euclidean distance
	restricted function class (many basis functions needed for complicated approximations)
	benefits from clustered structure in the data